Author: Ulugbek Norbutaev
Id: 405293
Date: 28 May 2026
Category: Informational

ASSIGNMENT I
PATTERN MATCHING ALGORITHMS

Status of This Document

This document is an informational submission prepared for university coursework. It describes the design, implementation, and empirical evaluation of six exact pattern matching algorithms, two wildcard matchers, and a two-dimensional Rabin-Karp matcher.

This document does not specify an Internet Standard.

ABSTRACT

Pattern matching locates all occurrences of a pattern P of length m inside a text T of length n. This document describes and evaluates six algorithms: Brute Force, Sunday, Knuth-Morris-Pratt (KMP), a Finite State Machine (FSM) matcher, Rabin-Karp, and Gusfield's Z-Algorithm. Wildcard extensions supporting "?" and "*" tokens are implemented for Brute Force and Sunday. A two-dimensional Rabin-Karp matcher is provided for Part 3. Empirical benchmarks on Moby Dick (Project Gutenberg) with text sizes from 10,000 to 1,000,000 characters confirm theoretical complexity predictions and demonstrate that no single algorithm dominates all input shapes.

TABLE OF CONTENTS

1. Introduction....................................3
2. Algorithm Descriptions..........................3
  2.1. Brute Force.................................3
  2.2. Sunday......................................3
  2.3. KMP...4
  2.4. FSM...4
  2.5. Rabin-Karp..................................4
  2.6. Gusfield Z-Algorithm........................4
3. Part 1A -- Benchmark Results....................5
4. Part 1B -- Wacky Races..........................6
5. Part 2 -- Wildcard Matching.....................7
6. Part 3 -- 2D Rabin-Karp.........................8
7. Conclusions.....................................8

8. References..9

1. INTRODUCTION

Pattern matching asks where a pattern P of length m appears inside a text T of length n. It is a fundamental operation in text editors, search engines, plagiarism detectors, compilers, antivirus scanners, and DNA sequence analysis tools.

This project implements six exact matchers, two wildcard matchers, and one two-dimensional hash-based matcher. All implementations are in JavaScript (Node.js). Benchmarks run against the full text of Herman Melville's "Moby Dick" obtained from Project Gutenberg (https://www.gutenberg.org/files/2701/2701-0.txt) and cached locally to ensure reproducibility.

All exact one-dimensional matchers share the signature:

```
algorithm(text, pattern) -> Array of zero-based match indices
```

Wildcard matchers return a Boolean. The 2D matcher returns an object `{ found, position }`.

Development was assisted by AI tools (Claude by Anthropic, ChatGPT by OpenAI, GitHub Copilot by GitHub/Microsoft) and web resources (Google Search, Google Scholar) [AI]. All implementations and analysis are the author's own.

2. ALGORITHM DESCRIPTIONS

2.1. Brute Force

Category: Exact matching baseline

Core idea: Tries every possible alignment of the pattern in the text and compares characters from left to right. It does no preprocessing and is easy to reason about.

Time: O(1) preprocessing. O(n*m) worst case, often better on random text.
Space: O(1) besides the output array.
Best: Most windows fail on the first character.
Worst: Text and pattern share long repeated prefixes.

2.2. Sunday

Category: Heuristic bad-character shift method

Core idea: After comparing a window, looks at the character immediately after that window. Shifts the pattern to align that character with its rightmost occurrence in the pattern. If absent, skips the whole window.

Time: O(m + alphabet) preprocessing. Average often sublinear; O(n*m) worst case.

Space: O(alphabet) for the shift table.

Best: After-window characters absent from pattern yield m+1 jumps.

Worst: Highly repetitive inputs cause tiny shifts and many comparisons.

2.3. KMP (Knuth-Morris-Pratt)

Category: Exact linear-time prefix-function algorithm

Core idea: Builds a prefix/failure function f[i] = length of the longest proper prefix of P[0..i] that is also a suffix. On mismatch after j matches, resumes at f[j-1] instead of restarting.

Time: O(m) preprocessing. O(n) search. O(n+m) total.

Space: O(m) for the prefix table.

Best: Patterns with repeated structure where naive search re-checks characters.

Worst: Still linear; fallback transitions add a constant factor.

2.4. FSM (Finite State Machine)

Category: Deterministic finite automaton exact matcher

Core idea: Builds states 0..m where state q means q pattern characters have been matched. Each text character follows one transition. State m is accepting.

Time: O(m * alphabet) preprocessing. O(n) search.

Space: O(m * alphabet), which can be large for wide alphabets.

Best: Repeated searches with the same pattern where preprocessing is amortized.

Worst: Large alphabets make the transition table expensive to build and store.

2.5. Rabin-Karp

Category: Rolling-hash exact matcher

Core idea: Hashes the pattern and each text window. A rolling update slides the hash one character right in O(1). Hash matches are verified by direct comparison to guard against collisions.

Time: O(n+m) average. O(n*m) worst case if every window requires verification.

Space: O(1) besides output.

Best: Few hash matches and therefore few full verifications.

Worst: Many true matches or collisions force repeated character-by-character checks.

2.6. Gusfield Z-Algorithm

Category: Exact linear-time Z-array method

Core idea: Builds S = P + separator + T and computes, for every position, how many characters match the prefix of S. A Z value of at least m after the separator indicates a full match.

Time: O(n+m) to build and scan the Z-array.

Space: O(n+m) for the combined string and Z-array.

Best: One-shot exact search or prefix-analysis tasks.

Worst: Still linear, but always allocates and scans the full combined string.

3. PART 1A -- BENCHMARK RESULTS (RT VS TEXT LENGTH)

Text slices of approximately 10,000 / 50,000 / 100,000 / 200,000 / 500,000 / 1,000,000 characters are taken from the beginning of the cached Moby Dick text. Two patterns are tested:

Small pattern : "the White Whale" (15 characters, first exact
occurrence in the book)
Large pattern : 300-character substring starting at position 20,000

Each (algorithm, size, pattern) triple is measured three times after one warm-up call. The median of the three measured runs is recorded to reduce sensitivity to garbage-collection pauses and OS scheduling.

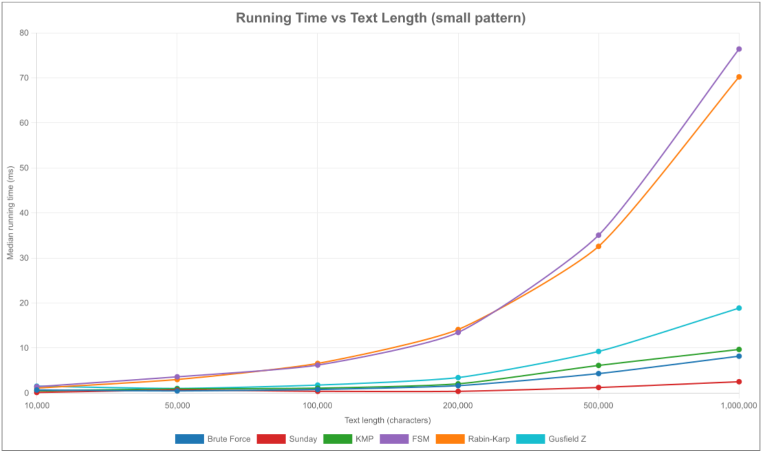

Figure 1. Running time vs text length for the small pattern.

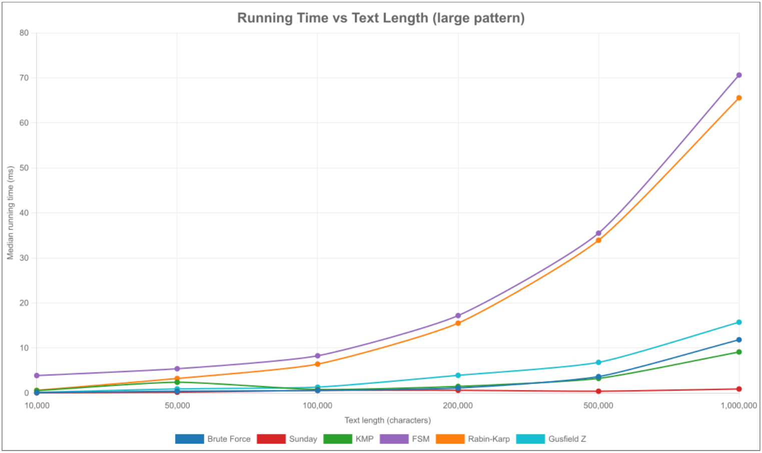

Figure 2. Running time vs text length for the large pattern.

small pattern: Sunday had the lowest total median time, while FSM had the highest. large pattern: Sunday had the lowest total median time, while FSM had the highest. Across increasing text sizes, the linear-time methods grow close to linearly, while methods that compare many characters per alignment become visibly more sensitive to pattern length and input structure.

4. PART 1B -- WACKY RACES

Adversarial inputs are constructed to show that no single algorithm wins on all input shapes. Each race text is at least 100 KB. Each algorithm is warmed up once, then timed for five runs; the median is recorded. A ratio >= 2.0x is required.

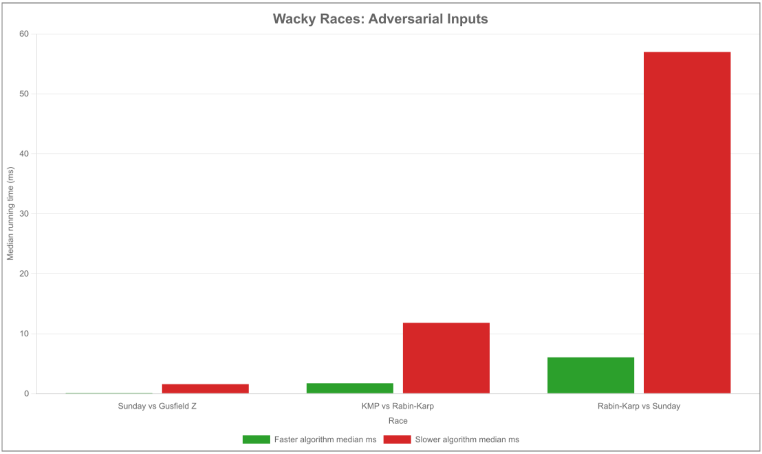

Figure 3. Median execution times for each race.

4.1. Sunday >= 2x faster than Gusfield Z

T[:100] : cccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccccc
cccccccccccccccc
Pattern : aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaab

Algorithm	**Median Time**	**Matches**
Sunday	0.008 ms	0
Gusfield Z	1.566 ms	0

Ratio: 188.73x. The text character after each Sunday window is never in the pattern, so Sunday jumps by m + 1. Gusfield still builds the full Z-array over pattern + separator + text.

4.2. KMP >= 2x faster than Rabin-Karp

T[:100] : aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaa
Pattern : aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa

Algorithm	**Median Time**	**Matches**
KMP	1.716 ms	102273
Rabin-Karp	11.800 ms	102273

Ratio: 6.88x. Every window is a hash hit for Rabin-Karp, so it must verify many long matches. KMP keeps a prefix-state machine and moves through the text linearly.

4.3. Rabin-Karp >= 2x faster than Sunday

```
   T[:100] : aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaa
   Pattern : aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaab
```

Algorithm	Median Time	Matches
Rabin-Karp	6.042 ms	0
Sunday	56.962 ms	0

Ratio: 9.43x. Sunday compares almost the whole pattern before the final mismatch and then shifts only two positions. Rabin-Karp rolls one hash per position and almost never verifies.

5. PART 2 -- WILDCARD MATCHING

Wildcard patterns support three token types:

?	Matches exactly one arbitrary character.
*	Matches zero or more arbitrary characters.
\? * \\	Escaped tokens are treated as literals.

5.1. Brute Force Wildcard

The pattern is tokenized into literals, "any_one" tokens (?), and "any_many" tokens (*). A recursive backtracking matcher tries each possible expansion of * lazily (zero characters first, then one more on backtrack). The search function wraps the core matcher so that the pattern is allowed to match any substring of the text, behaving as though implicit * tokens surround the pattern.

5.2. Sunday Wildcard

The token stream is split on * tokens, producing fixed segments. Each segment is searched left to right with a Sunday-style bad-character shift table. Question marks are handled conservatively: any text character is a valid match at a ? position, so the shift table must not skip over positions that could satisfy ?. Segments must appear in non-overlapping, strictly increasing order; * tokens permit arbitrary text gaps between segments.

5.3. Correctness Test Results

Case	Pattern	Expected	Brute Force	Sunday	Status
basic literal	world	true	true	true	PASS
question mark	h?llo	true	true	true	PASS
star consumes middle	h*o	true	true	true	PASS
escaped question	he\?lo	true	true	true	PASS
escaped star	he*lo	true	true	true	PASS
escaped backslash	he\\lo	true	true	true	PASS
no match	world	false	false	false	PASS
match at start	start*	true	true	true	PASS
match at end	*end	true	true	true	PASS
star matches empty	beau*tiful	true	true	true	PASS
match in middle	hello	true	true	true	PASS
combined wildcards	h?llo*world	true	true	true	PASS
combined no match	a*d	false	false	false	PASS

Large-text timing used 110010 characters
with pattern n*Z?nd.
Brute Force: 1.136 ms.
Sunday: 2.287 ms.

6. PART 3 -- 2D RABIN-KARP

Given an M x N picture of arbitrary items and an integer K, the function checks whether the top-right K x K block appears anywhere else in the picture.

Signature:

```
rabinKarp2D(picture, K)
-> { found: boolean, position: { row, col } | null }
```

6.1. Item Encoding

Picture items may be of any type. Before hashing, each unique item is mapped to a positive integer code using a Map. Hashing then operates on integer codes.

6.2. Two-Pass Hashing

Pass 1 (horizontal): For every row, compute rolling hashes of all length-K windows. Store one hash per (row, col) pair.

Pass 2 (vertical): For every column-window position col, maintain a rolling vertical hash over the K row hashes above. Roll the hash down each row in O(1) by subtracting the departing row hash and adding the arriving row hash.

6.3. Verification

Hash matches are verified by direct K x K cell comparison. Correctness does not depend on hash uniqueness; it depends only on the verification step.

6.4. Hash Arithmetic

The bitwise mask `& ((1 << 31) - 1)` is used in place of modulo division for speed. The mask value 2^31 - 1 is a Mersenne prime but the operation is a simple bit truncation, not true modular arithmetic. Collisions are possible but rare; each is resolved by direct comparison, so the result is always correct.

6.5. Complexity

```
Horizontal pass : O(M * N)
Vertical pass   : O(M * N)
Verification    : O(K^2) per hash hit, expected O(1) hits
Total expected  : O(M * N)
Space           : O(M * (N - K + 1)) for the row hash table
```

7. CONCLUSIONS

Brute Force is the correct baseline: zero preprocessing, simple reasoning, and often acceptable performance on random text.

Sunday performs well in practice when its shift table produces large jumps, but degrades to O(n*m) on repetitive inputs.

KMP and FSM both guarantee O(n+m) worst-case search time. KMP is usually preferred for space efficiency; FSM is attractive when the same pattern is searched many times and the transition table can be constructed once and reused.

Rabin-Karp excels when searching for multiple patterns (compare all hashes in one pass) and extends naturally to two-dimensional matching. Its weakness is degenerate inputs with many true matches or collisions.

Gusfield's Z-Algorithm is a clean, self-contained linear-time exact matcher. Its Z-array also solves broader prefix-analysis problems, making it more general than a dedicated search routine.

No single algorithm is best for all inputs. The wacky-race experiments confirm this: each of Sunday, KMP, and Rabin-Karp can be made to win or lose by at least 2x depending on the structure of the text and pattern.

8. REFERENCES

The following sources were consulted during the design, implementation, and analysis of this project.

[CLRS] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C.
Introduction to Algorithms, 4th ed. MIT Press, 2022.
(KMP, Rabin-Karp, FSM string matcher -- Chapter 32)

[GUS] Gusfield, D.
Algorithms on Strings, Trees, and Sequences.
Cambridge University Press, 1997.
(Z-Algorithm -- Chapter 1)

[CL] Crochemore, M. and Lecroq, T.
Handbook of Exact String Matching Algorithms.
King's College London Publications, 2004.
(Brute Force, Sunday -- Chapters 1 and 8)

[AI] AI & Web Tools.
Development was assisted by AI tools (Claude by Anthropic, ChatGPT by OpenAI, GitHub Copilot by GitHub/Microsoft) and web resources (Google Search, Google Scholar).
All implementations and analysis are the author's own.